# 💻 เอกสารอธิบายโค้ดอย่างละเอียดทีละบรรทัดทุกไฟล์ คัมภีร์พูดได้ (Kumpee AI - พระไตรปิฎก AI)

คู่มือการอ่านโค้ดเชิงลึก: อธิบายการทำงาน ความหมาย และเหตุผลของการเขียนแต่ละบล็อกคำสั่ง

> 📌 วิธีการอ่านรายงานฉบับนี้
> เอกสารฉบับนี้ทำการถอดโค้ดจริงทุกไฟล์ในระบบ ได้แก่ `api/chat.js` , `api/tts.js` , `server.js` และ `public/app.js` โดยระบุเลขบรรทัด บล็อกคำสั่ง และอธิบายการทำงานในแต่ละส่วน เพื่อให้เข้าใจกลไกภายในทั้งหมดอย่างชัดเจน

## 1. อธิบายโค้ดไฟล์ `api/chat.js` (Vercel Serverless Function Proxy)

ทำหน้าที่เป็น API Proxy ปลอดภัยเชื่อมต่อกับ Anthropic Claude API บน Vercel Cloud Serverless

| บรรทัดที่ | โค้ดในไฟล์ | คำอธิบายการทำงานทีละบรรทัด |
|---|---|---|
| `1-7` | `const HARDCODED_CLAUDE_KEY = 'sk-ant-api03-...';`<br>`const DEFAULT_MODEL = 'claude-haiku-4-5-20251001';` | กำหนดค่าคงที่ API Key สำรองกรณีไม่ได้ตั้ง Environment Variables และกำหนดโมเดลมาตรฐานเป็น `claude-haiku-4-5-20251001` |
| `8-12` | `const MAX_MESSAGES = 20;`<br>`const MAX_MESSAGE_LENGTH = 6000;`<br>`const RATE_LIMIT_MAX_REQUESTS = 15;` | กำหนดขีดจำกัดความปลอดภัย: รับประวัติไม่เกิน 20 ข้อความ, ความยาวไม่เกิน 6,000 ตัวอักษรต่อข้อความ และจำกัดยิง API ไม่เกิน 15 ครั้ง/นาที |
| `15-25` | `const ALLOWED_ORIGINS = [`<br>`'https://kumpee-ai-psi.vercel.app',`<br>`/^https:\/\/kumpee-.*\.vercel\.app$/`<br>`];` | สร้างรายการ Whitelist โดเมนที่อนุญาตให้เรียกใช้ API Proxy ได้ รองรับทั้ง Exact Domain Name และ Regular Expression สำหรับ Preview URLs |
| `28-70` | `const SYSTEM_PROMPT = `คุณคือ "คัมภีร์พูดได้"...`;` | กำหนดบทบาท Prompt หลักของระบบไว้ที่ฝั่ง Server เพื่อป้องกันไม่ให้ฝั่ง Client แอบส่ง Prompt มาแก้ไขเอง |
| `73-95` | `function isRateLimited(ip) { ... }` | ฟังก์ชันตรวจความถี่คำขอของผู้ใช้ตาม IP Address หากส่งเกิน 15 ครั้งใน 60 วินาที จะส่งค่า `true` เพื่อปฏิเสธการทำงาน |
| `98-106` | `function isOriginAllowed(origin) { ... }` | ฟังก์ชันตรวจเช็คว่า Header `Origin` ที่ยิงเข้ามา ตรงกับรายการที่ได้รับอนุญาตใน `ALLOWED_ORIGINS` หรือไม่ |
| `109-119` | `function sanitizeMessages(messages) { ... }` | กรองและทำความสะอาดประวัติข้อความ ตัดทอนข้อความที่ยาวเกิน 6,000 ตัวอักษร และกรอง role ให้เป็น `user` หรือ `assistant` เท่านั้น |
| `122-175` | `function generateSmartFallback(fullMsg) { ... }` | ระบบคำตอบสำรอง (Fallback): เมื่อ AI ขัดข้อง ฟังก์ชันนี้จะดึงข้อความพระสูตรอันดับ 1 จากบริบทมาจัดรูปแบบเป็นคำตอบภาษาไทยสวยงาม |
| `178-190` | `res.setHeader('Access-Control-Allow-Origin', ...);`<br>`if (req.method === 'OPTIONS') return res.status(204).end();` | ตั้งค่า CORS Headers สำหรับตอบกลับเบราว์เซอร์ และรองรับคำขอ Preflight (OPTIONS Request) ตามมาตรฐาน HTTP CORS |
| `192-205` | `if (origin && !isOriginAllowed(origin)) {`<br>`return res.status(403).json({ error: 'Origin not allowed' });`<br>`}` | ตรวจสอบ Origin หากคำขอมาจากโดเมนภายนอกที่ไม่ได้รับอนุญาต จะส่ง HTTP 403 Forbidden ปฏิเสธทันที |
| `207-216` | `if (isRateLimited(clientIP)) {`<br>`return res.status(429).json({ error: 'Too many requests' });`<br>`}` | เช็ค Rate Limit หากผู้ใช้ยิงคำขอถี่เกินไป จะส่ง HTTP 429 Too Many Requests ให้ผู้ใช้รอสักครู่ |
| `218-232` | `const messages = sanitizeMessages(bodyData.messages);` | แปลงคำขอเป็นวัตถุ JSON และส่งไปทำ Sanitization หากไม่พบข้อความที่ถูกต้อง จะคืนค่า 400 Bad Request |
| `234-265` | `const claudeResponse = await fetch('https://api.anthropic.com/v1/messages', { ... });` | ยิง HTTP POST Request ไปยัง Anthropic Claude API โดยส่ง `SYSTEM_PROMPT` ฝั่ง Server และใช้โมเดล `claude-haiku-4-5-20251001` |

| | | |
|---|---|---|
| `267-275` | `const fallbackText = generateSmartFallback(lastUserMsg); return res.status(200).json({ content: [{ text: fallbackText }], _fallback: true });` | กรณี Claude API ส่งข้อผิดพลาดกลับมา ระบบจะทำงานบนเส้นทาง Fallback อัตโนมัติ พร้อมส่งคีย์ `_fallback: true` แจ้งเตือนฝั่งหน้าเว็บ |

## 2. อธิบายโค้ดไฟล์ `api/tts.js` (Vercel Serverless TTS Stream Proxy)

ทำหน้าที่เป็นพร็อกซีเปลี่ยนข้อความภาษาไทยให้กลายเป็นสตรีมไฟล์เสียง MP3 จาก Google Translate TTS

| บรรทัดที่ | โค้ดในไฟล์ | คำอธิบายการทำงานทีละบรรทัด |
|---|---|---|
| `6-18` | `res.setHeader('Access-Control-Allow-Origin', '*');`<br>`if (req.method === 'OPTIONS') return res.status(204).end();` | กำหนด CORS Headers อนุญาตให้หน้าเว็บเรียกใช้ไฟล์เสียงได้ และรองรับ HTTP OPTIONS Request |
| `20-24` | `const text = req.query?.text;`<br>`if (!text) return res.status(400).json({ error: 'Missing text' });` | อ่านพารามิเตอร์ `text` จาก Query String เช่น `/api/tts?text=สวัสดี` หากไม่มีจะคืนค่า 400 Bad Request |
| `26-34` | `const encodedText = encodeURIComponent(text.substring(0, 200));`<br>`const googleTTSUrl = `https://translate.google.com/...`;` | สกัดข้อความไม่เกิน 200 ตัวอักษร นำไปเข้ารหัส URL (URI Encoding) และสร้าง URL ดึงเสียงอ่านภาษาไทย ( `tl=th` ) จาก Google TTS |
| `35-45` | `const ttsResponse = await fetch(googleTTSUrl, {`<br>`headers: { 'User-Agent': 'Mozilla/5.0...' } });` | ส่งคำขอ HTTP GET ไปยังเซิร์ฟเวอร์ Google Translate โดยแนบ User-Agent ปลอมเพื่อป้องกันไม่ให้ Google บล็อกสตรีมเสียง |
| `47-50` | `const audioBuffer = await ttsResponse.arrayBuffer();`<br>`res.setHeader('Content-Type', 'audio/mpeg');`<br>`return res.send(Buffer.from(audioBuffer));` | แปลงสตรีมเสียงเป็น Binary ArrayBuffer แล้วส่งคืนให้ผู้ใช้ในรูปแบบไฟล์เสียง `audio/mpeg` (MP3) พร้อมตั้งค่าแคช 24 ชั่วโมง |

## 3. อธิบายโค้ดไฟล์ `server.js` (Local Node.js HTTP Server)

ทำหน้าที่บริการเว็บและประมวลผลคำขอเมื่อเปิดใช้งานในเครื่องคอมพิวเตอร์ผ่าน `node server.js`

| บรรทัดที่ | โค้ดในไฟล์ | คำอธิบายการทำงานทีละบรรทัด |
|---|---|---|
| `5-9` | `const http = require('http');`<br>`const https = require('https');`<br>`const fs = require('fs');` | โหลด Native Node.js Modules สำหรับสร้าง HTTP Server, ยิงคำขอ HTTPS ภายนอก และอ่านไฟล์จากดิสก์ |
| `13-16` | `const PORT = 8080;`<br>`const CLAUDE_MODEL = 'claude-haiku-4-5-20251001';` | กำหนดพอร์ตทำงานเป็น `8080` และกำหนดใช้โมเดล Claude รุ่นปัจจุบันให้ตรงกับเซิร์ฟเวอร์คลาวด์ |
| `33-60` | `const server = http.createServer((req, res) => { ... });` | สร้าง HTTP Server ตรวจจับ Request URL: หากเข้า `/api/chat` เรียก `handleChatProxy()`, เข้า `/api/tts` เรียก `handleTTSProxy()`, นอกนั้นส่งให้ `serveStaticFile()` |
| `65-105` | `function generateSmartFallback(userMessage) { ... }` | ระบบคำตอบสำรองในเครื่อง: วิเคราะห์คำถามและสกัดพระสูตรอันดับ 1 จากบริบท นำมาตอบแทนเมื่อไม่มีอินเทอร์เน็ต |
| `110-194` | `function handleChatProxy(req, res) { ... }` | รับ Body จาก Client ตรวจสอบ Rate Limit และส่งคำขอผ่าน `https.request` ไปยัง Claude API พร้อม `SYSTEM_PROMPT` ฝั่ง Server |
| `199-281` | `function handleTTSProxy(req, res) { ... }` | ดึงเสียงอ่านจาก Google Translate TTS รองรับการติดตาม HTTP Redirect 301/302 อัตโนมัติ แล้วสตรีมไฟล์ MP3 กลับไปให้เบราว์เซอร์ |
| `286-322` | `function serveStaticFile(req, res) { ... }` | อ่านและส่งมอบไฟล์ Static (`index.html`, `index.css`, `app.js`) จากโฟลเดอร์ `public/` พร้อมระบบป้องกัน Directory Traversal |
| `327-350` | `server.listen(PORT, '0.0.0.0', () => { ... });` | เริ่มต้นรันเซิร์ฟเวอร์บนพอร์ต 8080 พร้อมค้นหา IP ในเครือข่ายท้องถิ่น (LAN IP) พิมพ์บอกบนคอนโซล เพื่อให้เครื่องอื่นในวง Wi-Fi เข้าใช้งานได้ |

## 4. อธิบายโค้ดไฟล์ `public/app.js` (Client Front-end Application)

สคริปต์หลักที่รันบนเว็บเบราว์เซอร์ของผู้ใช้ จัดการทำงานทั้งหมดตั้งแต่ UI, คลังข้อมูลพระสูตร, อัลกอริทึมค้นหา และการแสดงผลมหาปเทส 4

| บรรทัดที่ | โค้ดในไฟล์ | คำอธิบายการทำงานทีละบรรทัด |
|---|---|---|
| `9-20` | `const CONFIG = { SHEET_CSV_URL: 'https://docs.google.com/...', CHAT_API_URL: '/api/chat', CACHE_KEY: 'tipitaka_data_v3' };` | กำหนดค่าคงที่กลาง: ลิงก์ Google Sheets CSV (932 รายการ), Endpoint API, คีย์แคช `v3` และความเร็วการพิมพ์ข้อความ |
| `65-125` | `function parseCSV(text) { ... }` | ฟังก์ชันตัดคำและแยกคอลัมน์จากไฟล์ CSV รองรับข้อความยาวที่มีเครื่องหมายคำพูดคร่อม (Quoted text) และการเว้นบรรทัดในเซลล์ |
| `130-164` | `async load(onProgress) { ... }` | โหลดคลังพระสูตร: ตรวจสอบแคชใน `localStorage` ก่อน หากไม่มี จะดึงไฟล์จาก Google Sheets นำมาแปลงเป็นวัตถุพระสูตร |
| `206-235` | `_setCache(rows) { ... }` | บันทึกแคชอัจฉริยะ (localStorage Optimization): เลือกบันทึกเฉพาะ 11 ฟิลด์ที่ใช้จริง และตัดทอนเนื้อหาเหลือ 1,000 ตัวอักษรเพื่อประหยัดพื้นที่ 70% |
| `245-310` | `search(query) { ... }` | อัลกอริทึมค้นหาพระสูตร (Weighted Search Ranking): คำนวณคะแนนความเกี่ยวข้อง โดยให้คะแนนพิเศษเมื่อตรงกับชื่อพระสูตร คำสอนสำคัญ หรือคีย์เวิร์ดภาษาไทย |
| `315-385` | `async chat(userMessage) { ... }` | ค้นหาพระสูตรที่เกี่ยวข้อง (Round 1 RAG) รวมไม่เกิน 4,000 ตัวอักษร แนบส่งไปหา `/api/chat` จากนั้นนำคำตอบ AI ที่ได้ไปค้นหาซ้ำในคลังข้อมูล (Round 2 Verification) |
| `388-425` | `_generateVerdict(contextSuttas, verifiedSuttas, isFallback)` | คำนวณสร้างการ์ดมหาปเทส 4 (Verdict Object): ประเมินว่าคำตอบ AI `สอดคล้อง` กับพระสูตรในคลัง หรือ `ไม่พบหลักฐาน` ยืนยัน |
| `516-603` | `speak(text) { ... }` | ตัดเครื่องหมายสัญลักษณ์และอิโมจิออกจากข้อความ แบ่งข้อความตามจุดจบประโยค แล้วส่งเล่นไฟล์เสียง MP3 จาก `/api/tts` ทีละท่อน |
| `633-655` | `startListening() { ... }` | เปิดใช้งาน `SpeechRecognition` ถอดเสียงพูดภาษาไทยของผู้ใช้เป็นข้อความ นำไปวางในช่องพิมพ์แชตอัตโนมัติ |
| `739-805` | `showReference(contextSuttas, verifiedSuttas, verdict)` | วาดแสดงผลพาเนลคัมภีร์อ้างอิง: แสดง Verdict Card ด้านบนสุด ตามด้วยกรอบสีเขียว (พระสูตรยืนยันรอบ 2) และกรอบสีทอง (พระสูตรบริบทรอบ 1) |
| `809-985` | `NavigatorModule = { ... }` | จัดการอวาตาร์ 2D: ตัดพื้นหลังขาวด้วย HTML5 Canvas, เล่นแอนิเมชันขยับปากพูดแบบ Lip-sync Loop (สลับสภาวะนิ่ง/พูดทุก 150ms) และรองรับการลากวาง (Draggable) |
| `989-1144` | `ChatModule = { ... }` | จัดการ UI แชต: แสดงข้อความต้อนรับ, ปุ่มชิปคำถามแนะนำ, พิมพ์ข้อความคำตอบแบบ Typewriter Animation และป้ายกำกับที่มาข้อความ |
| `1149-1430` | `App = { ... }` | Controller ศูนย์กลาง: โหลดข้อมูลพระสูตรตอนเปิดเว็บ, ผูก Event Listeners สำหรับปุ่มส่งคำถาม ปุ่มไมโครโฟน ปุ่มหยุดเสียงอ่าน และป๊อปอัปตั้งค่าเสียง |